

RELEASE DOCUMENT · FIRST PUBLIC RELEASE

Aeza Codex

System Design Document

Version 0.1.0

Document ID AC-SDD-001

Date 17 August 2026

Classification Public (MIT License)

Self-hosted RAG chatbot for a business knowledge base.

Architecture, data, security, and deployment baseline.

Contents

1. Purpose and scope
2. System context
3. Logical architecture
4. Runtime configuration
5. HTTP surface
6. Authentication and session
7. Data design
8. Ingest and retrieval
9. Security controls
10. Deployment
11. Component directory
12. Constraints and residual risks
13. Release 0.2.0 (planned)
14. Document history

1. Purpose and scope

1.1 Purpose

Aeza Codex is a self-hosted retrieval-augmented generation (RAG) chatbot for a business knowledge base. An administrator indexes documents and optional website pages. Visitors ask questions in a branded chat UI. Answers are produced from retrieved chunks, or from structured catalog and product data extracted from Excel, not from unconstrained general knowledge.

1.2 In scope (v0.1.0)

- Customer chat at /chat and administrator console at /admin
- Ingest of PDF, DOCX, TXT, XLSX/XLS, and a single public HTTP(S) page
- OpenAI-compatible embeddings and chat, including NVIDIA NIM
- FAISS inner-product index with on-disk or Vercel Blob persistence
- Cookie-based administrator authentication
- Embeddable chat iframe

1.3 Out of scope (v0.1.0)

- Multi-instance vector search (Qdrant) — scheduled for v0.2.0
- Multi-worker process models
- Conversation memory across turns
- Site-wide crawl and multi-tenant SaaS control plane

1.4 Design principles

- Grounded answers: the language model may use only retrieved or structured context.
- Single-process truth: the FAISS index is process memory; one worker is supported.
- Downward dependencies: HTTP to services to ingestion/RAG to storage.
- Replaceable LLM: OpenAI client against a configurable base URL.

2. System context

Figure 1. System context
Aeza Codex v0.1.0 — external actors and dependencies

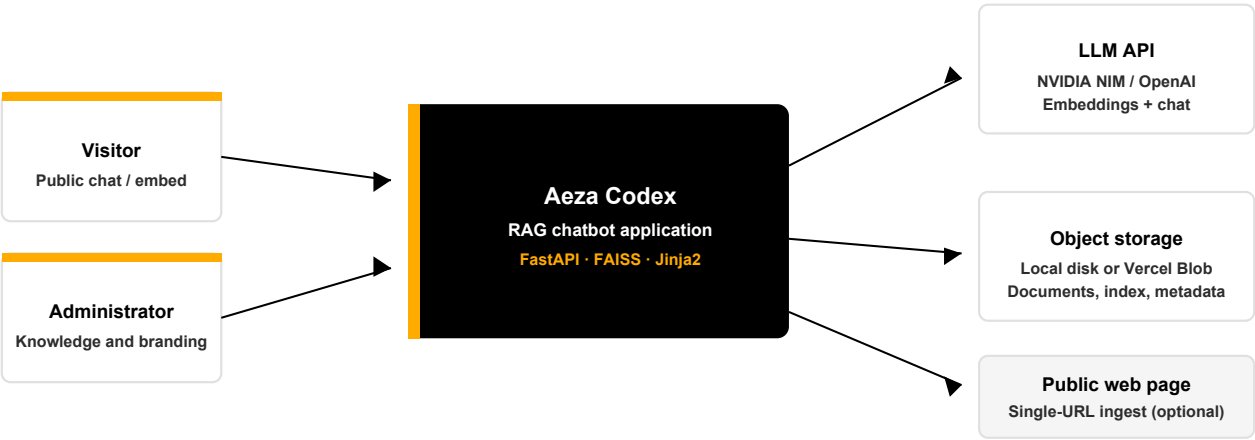


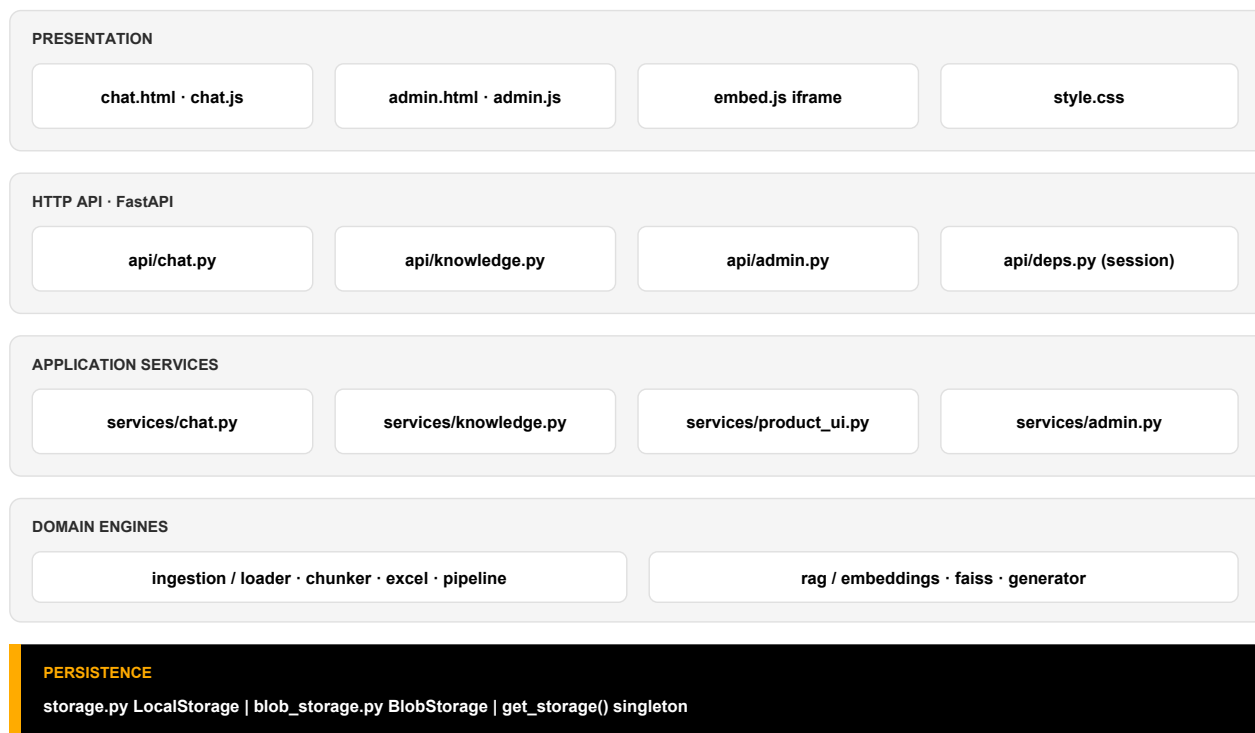
Figure 1. System context — external actors and dependencies.

Actor	Interaction
Visitor	Public /chat or iframe embed. Receives streamed or complete replies and optional product cards.
Administrator	/admin after login. Uploads sources, reindexes, sets branding, changes credentials.
LLM API	NVIDIA NIM or OpenAI-compatible API. Embeddings and chat completions.
Object storage	Local filesystem or Vercel Blob. Files, FAISS snapshot, mapping, catalogs, credentials.

3. Logical architecture

Figure 2. Logical architecture

Layered design. Dependencies point downward only.

*Figure 2. Logical architecture — layers depend downward only.*

Presentation is server-rendered Jinja2 plus static CSS/JS. The chat client prefers POST `/api/chat/stream` and falls back to POST `/api/chat`. FastAPI routers sit under `/api/chat`, `/api/knowledge`, and `/api/admin`. Services orchestrate ingest and query. Storage is selected by `get_storage()` as LocalStorage or BlobStorage.

4. Runtime configuration

Source: `app/config.py`, class `Settings`. Environment file `.env`. Unknown keys are ignored.

Setting	Env var	Default	Notes
environment	ENVIRONMENT	development	production requires OPENAI_API_KEY at startup
openai_api_key	OPENAI_API_KEY	empty	Required for ingest and RAG chat
openai_base_url	OPENAI_BASE_URL	empty	NVIDIA NIM or OpenAI-compatible URL
embedding_model	EMBEDDING_MODEL	text-embedding-3-small	NVIDIA: nvidia/nv-embedqa-e5-v5
chat_model	CHAT_MODEL	gpt-4o-mini	NVIDIA: openai/gpt-oss-120b
chat_max_tokens	CHAT_MAX_TOKENS	4096	On Vercel, effective cap is min(value, 1024)
data_dir	DATA_DIR	data	Local object root
rag_top_k	RAG_TOP_K	5	Default retrieval width
rag_min_score	RAG_MIN_SCORE	0.25	Minimum inner-product score after L2 normalize
admin_username	ADMIN_USERNAME	admin	Bootstrap username

Setting	Env var	Default	Notes
admin_password	ADMIN_PASSWORD	empty	Required for first login
storage_backend	STORAGE_BACKEND	empty	Honored only off Vercel: local or blob
blob_read_write_token	BLOB_READ_WRITE_TOKEN	empty	Required for Blob backend

On Vercel, storage is blob if a token exists, otherwise local under /tmp/aeza-codex. STORAGE_BACKEND is not consulted on Vercel. LLM HTTP timeout is 50 seconds on Vercel and 120 seconds otherwise.

5. HTTP surface

5.1 Pages and chat

Method	Path	Access	Description
GET	/	Public	Redirect to /chat
GET	/chat	Public	Customer UI; embed=1 fills iframe
GET	/admin	Public HTML	Console; APIs require a session
GET	/health	Public	Process status JSON
POST	/api/chat	Public	JSON message to reply, sources, ui
POST	/api/chat/stream	Public	SSE meta, token, done, error

5.2 Knowledge and administration

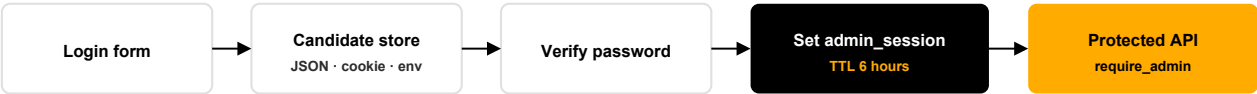
Method	Path	Access	Description
POST	/api/knowledge/upload	Admin	Multipart file ingest
POST	/api/knowledge/url	Admin	Single public page
GET	/api/knowledge/documents	Public	List source metadata
DELETE	/api/knowledge/documents/{id}	Admin	Remove source and vectors
POST	/api/admin/login	Public	Sets session cookie
PUT	/api/admin/credentials	Admin	Change username and password
GET	/api/admin/settings	Public	Branding for chat UI
PUT	/api/admin/settings	Admin	Save branding
POST	/api/admin/reindex	Admin	Rebuild FAISS from stored files

session_id is generated if omitted and echoed only. The model does not receive prior turns. Accepted uploads: .pdf, .docx, .txt, .xlsx, .xls. Oversize returns HTTP 413.

6. Authentication and session

Figure 6. Administrator authentication

HttpOnly cookies. HMAC session. PBKDF2-HMAC-SHA256 passwords (200,000 iterations).



admin_session payload: username | expiry | HMAC-SHA256(password_hash, username|expiry). Cookie flag Secure is set when ENVIRONMENT=production or VERCEL is present.

admin_creds (optional, 1 year) stores the hashed credential record so login survives ephemeral /tmp when Blob is not configured. Signed with ADMIN_PASSWORD.

Public HTML at GET /admin. Mutating knowledge and branding APIs require a valid session. Chat APIs are unauthenticated by design.

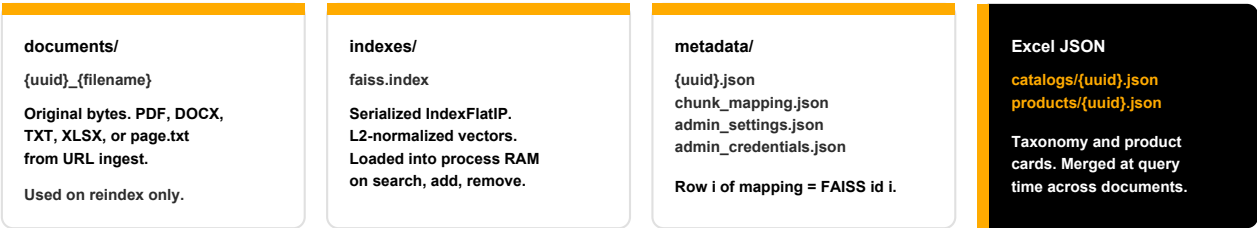
Figure 6. Administrator authentication — HttpOnly cookies and PBKDF2 passwords.

Passwords are PBKDF2-HMAC-SHA256 (200,000 iterations, 16-byte hex salt) in admin_credentials.json. Cookie admin_session lasts 6 hours. Cookie admin_creds lasts 1 year and is signed with ADMIN_PASSWORD so login can survive ephemeral /tmp. Both cookies are HttpOnly, Path=/, SameSite=Lax. Secure is set in production or on Vercel. Chat APIs are unauthenticated by design.

7. Data design

Figure 4. Data stores

Same relative layout on local disk and Vercel Blob (prefix aeza-codex/).



v0.1 persistence model

The FAISS file is a snapshot of process memory, not a multi-writer database. Concurrent instances can diverge until reload.
Local recommended path: DATA_DIR (default data/). Vercel: BLOB_READ_WRITE_TOKEN required for durability; otherwise /tmp/aeza-codex.
v0.2 replaces faiss.index and chunk_mapping.json as the live search store with Qdrant. Object storage remains for original files.
chunk_mapping.json fields: document_id, chunk_index, text, filename, chunk_type (optional).

Figure 4. Data stores — identical relative paths on disk and Blob.

Artifact	Path	Role
Original file	documents/{uuid}_{filename}	Reindex input
FAISS snapshot	indexes/faiss.index	Serialized IndexFlatIP
Chunk map	metadata/chunk_mapping.json	FAISS row to text
Document meta	metadata/{uuid}.json	Status and chunk count
Catalog / products	metadata/catalogs products/{uuid}.json	Excel taxonomy and cards
Branding / creds	metadata/admin_*.json	Settings and password hash

8. Ingest and retrieval

Figure 3. Ingest and retrieval

Upper path: index time. Lower path: query time.

INGEST



QUERY



Excel hybrid ingest (additional artifacts)

Product sheets produce three chunk types (product, category, catalog overview) plus structured JSON.

Catalog JSON drives deterministic category answers. Product JSON drives cards, compare, and price sort.

These shortcuts execute before FAISS. If they do not match, retrieval and the LLM run as shown above.

Figure 3. Ingest (index time) and retrieval (query time).

Ingest assigns a UUID, stores original bytes, extracts text, chunks (512/64 for prose; Excel hybrid product/category/catalog chunks), embeds with `input_type=passage`, then `faiss.add`. Query tries catalog shortcuts, then product UI, then embed query plus FAISS search with `min_score 0.25`, then the LLM with `RAG_SYSTEM_PROMPT`. Vectors are L2-normalized IndexFlatIP. `ensure_loaded` hydrates RAM from storage when empty or stale (20 s fingerprint).

9. Security controls

Control	Implementation
Administrator authentication	Cookie session, PBKDF2, HMAC
Upload size and type	10 MB default; extension allow-list
URL ingest SSRF	Scheme, DNS, private IP denial, redirect re-check
Secrets	API keys and admin password from environment
Cookie flags	HttpOnly, SameSite=Lax, Secure in production/Vercel
LLM grounding	Prompt plus score threshold; not a confidentiality guarantee

Public by design: chat API, document list, branding GET. Do not index confidential material on a public deployment without additional access control (not provided in v0.1.0).

10. Deployment

Figure 5. Deployment topologies

v0.1 supported topology is a single long-running process.

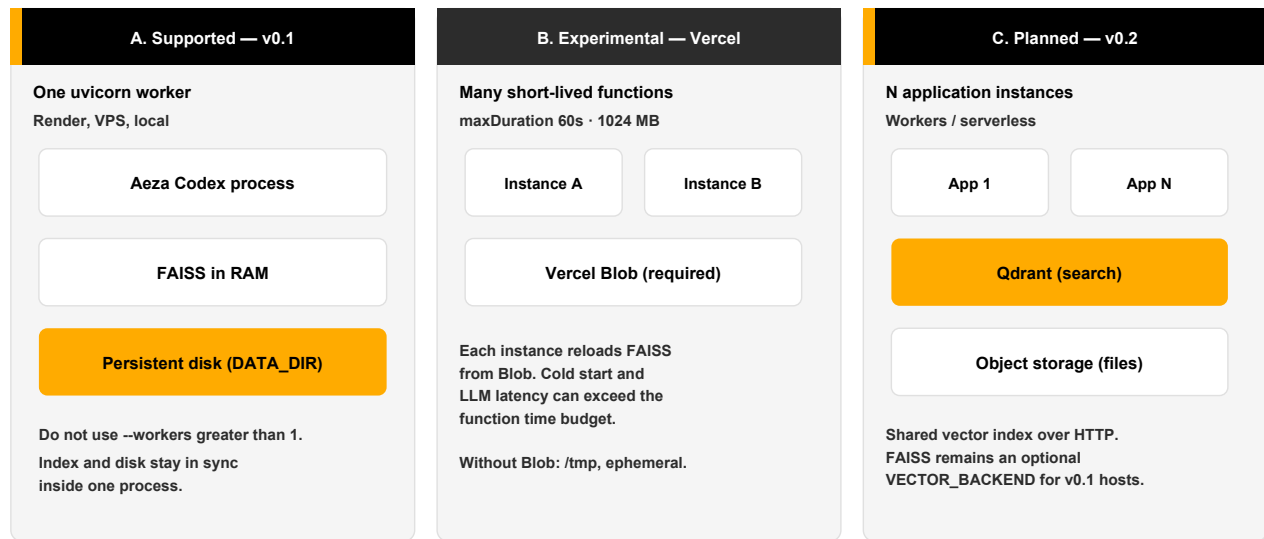


Figure 5. Deployment topologies — supported single process, experimental Vercel, planned Qdrant.

Supported: one uvicorn process, STORAGE_BACKEND=local, persistent DATA_DIR. Do not set workers greater than 1. Experimental Vercel: Fluid, maxDuration 60 s, 1024 MB; Blob token required for durability. Entrypoint app.main:app. Python 3.12 or newer.

```
uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

11. Component directory

app/main.py application · app/config.py settings · app/api chat, knowledge, admin, deps · app/services chat, knowledge, product_ui, admin · app/ingestion pipeline, loader, chunker, excel · app/rag embeddings, faiss, generator · app/storage LocalStorage and BlobStorage · app/templates and app/static.

12. Constraints and residual risks

- FAISS is not a database. The live index is RAM. Concurrent writers can overwrite snapshots.
- No multi-turn memory. Each request is independent.
- Vercel is not the primary host. Function time limits and instance isolation conflict with index load plus LLM.
- Without Blob, /tmp knowledge and credentials vanish with the instance.
- Health vector counts are in-memory and may read zero until hydrate.
- primary_color is stored but not applied by the chat client in v0.1.0. Sources are not rendered in the UI. No automated tests in this release.

13. Release 0.2.0 (planned, not implemented)

Item	Intent
Qdrant	Shared vector store over HTTP; payload holds chunk text
VECTOR_BACKEND=faiss qdrant	Preserve v0.1 single-process installs
Multi-worker / multi-instance	Safe once search is not process-local RAM

Item	Intent
Delete by document_id	Avoid full index rewrite
Rate limit on /api/chat	Public endpoint protection
Optional async ingest	Large Excel without blocking chat

14. Document history

Version	Date	Description
0.1.0	2026-08-17	First public release. FAISS persistence, cookie admin, Excel hybrid ingest, Vercel experimental path.